PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

JACKFRUIT -MINIPROJECT

SOFTWARE ENGINEERING -PROJECT GUIDELINES

Subject Code : UE24CS341A

Purpose: As part of the software engineering course, students are expected to do a project incorporating all stages of software development life cycle. As the units will be taught in the classes, phase wise inclusion needs to be done.

Outcome: Synopsis and Complete Project Report (Formats attached for reference)

- Documentation is equally important as code
- Modular design, maintainability, debuggability, extensible design
- Agile methodology only
- Github (Jira or similar tool) actively used for user stories, backlog etc
- CI/CD pipeline, code reviews, security validation mandatory
- Feel free to use LLMs for coding or anything else
    - But if you can't explain what you submitted, grade will be significantly affected
    - Each team will present not just features but code also. Must understand
- Evaluation method: LLM prompts, understanding, group dynamics, leadership, originality, novelty aspects
- Every member of the team must contribute to the implementation – git commits will be checked

Evaluation and complete Process: The details of how this project must be done and evaluated are as described below:



PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Guidelines:

- Each team responsible for developing a project Overall deliverables for each team are as follows:
  - As Development Team
    - Requirements Specification , Arch and Design specification
    - PRs, Unit test report showing code and branch coverage

1: Project team must identify an application case study ex: e-ticket/Banking/Recruitment System/Trading System. *(Discussion regarding inclusion of database and choice of PL required must be deliberated upon, also do* we allow projects like online games etc

2: The team must prepare a synopsis to enlist all the features of the chosen application.

3: Prepare a document consisting of requirements, planning, design, implementation, and testing details.

4: A report consisting of all the relevant documents must be submitted.

Project Team:

The project will be done by a group of 3/4 students. One or two teams in a class may be an exception with prior approval of the subject teacher. Teams must be among students belonging to the same section. Each team member needs to complete one full functionality. Weekly evaluations will be done based on individual performance and contribution made towards the project.

PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Stage	Deliverable	Student Learnings	Weightage
Requirements	SW Requirements Specification	Customer interaction, use case development, Building measurable and testable requirements	20
	Validation Spec	Testability of requirements, functional/security/non functional validation, pen testing --> Focused on Planning only	
Design	High level Architecture document	Use case development, Modular design, interface definition, UI/UX	7
	SW Design document	UML, API and ABI definition, Modular design, division of work within team	7
	Validation Spec update	Validation methodology, security validation, requirements traceability --> based on Arch and Design document	6
Implementation	Iterative design and implementation	Agile development, User task definition and mapping to user stories, code coverage, requirements traceability, Team interactions, stability of API and ABI Test case development	20
	Unit test coverage		5
	Validation Spec update		5
System Validation	Validation report + maintenance plan	Agile development, system view, usage driven validation, black/white box testing Product maintenance plan	15
Final demo and presentation	Working demo and 15mins presentation	Showing use cases, ability to highlight differentiation, showcase SDLC stages and output for each	15

Project Deliverables and Flow



PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Tools & Infrastructure

- Tools needed for each team:
  - Distributed version control: Git
  - Version control system: github Continuous Integration (CI): Jenkins
  - Continuous Deployment (CD): Jenkins
  - Containerization: docker
  - Testing tools: gtest, Junit, pytest etc
  - Bug tracking: Github (Jira or similar)
  - Collaboration and agile project management: Jira or similar
  - Static code analysis: SonarQube or similar
- Setup requirements and options
- Github https://github.com/education

PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Pick your Project Title

Project ID	Project Name	Project Scope
1	Naukri.com–Focused Automated Job Application and Resume Keyword Scheduler	Build a scheduler that parses a student's resume to extract key skills and job preferences, then automatically searches and applies to relevant fresh job postings on Naukri.com. The system should handle login, form filling, and application tracking while respecting site policies and rate limits.
2	Indeed India–Focused Automated Job Application and Resume Keyword Scheduler	Develop an automation tool that reads resume keywords and job criteria, then programmatically searches, filters, and applies to suitable entry-level jobs on Indeed India. Include features for scheduling runs, logging applications, and avoiding duplicate submissions.
3	LinkedIn Jobs–Focused Automated Job Application and Resume Keyword Scheduler	Create a system that extracts skills and roles from a resume and uses LinkedIn's job search (via API or browser automation) to find and apply to relevant postings. Focus on profile-based applications, connection requests to recruiters, and application status tracking.
4	Glassdoor–Focused Automated Job Application and Resume Keyword Scheduler	Implement a bot that scans Foundit for jobs matching resume-derived keywords, auto-fills application forms, and submits profiles. Add scheduling, error handling for CAPTCHAs or login issues, and a dashboard to review applied jobs.
5	Shine–Focused Automated Job Application and Resume Keyword Scheduler	Design an automation framework that matches resume skills with Shine job listings, auto-applies to eligible roles, and maintains a log of submissions. Include keyword weighting to prioritize high-relevance jobs and avoid spammy applications.
6	ZipRecruiter–Focused Automated Job Application and	Build a tool that parses resume content, searches TimesJobs for matching roles, and automates the application process with scheduled runs. Provide reporting on applied positions, response rates, and suggestions to improve resume keywords.



PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

	Resume Keyword Scheduler	
7	Wellfound–Focused Automated Job Application and Resume Keyword Scheduler	Develop a mobile-friendly automation system that uses resume data to find and apply to local and entry-level jobs on Apna. Handle profile creation, job matching, and application tracking while ensuring compliance with platform terms.
8	CareerBuilder–Focused Automated Job Application and Resume Keyword Scheduler	Create a scheduler that extracts candidate preferences from a resume and auto-applies to blue-collar and service-sector jobs on WorkIndia. Include location-based filtering, shift preferences, and a simple UI for students to monitor applications.
9	SimplyHired–Focused Automated Job Application and Resume Keyword Scheduler	Implement an automation tool targeting tech and startup jobs on Hirist, using resume keywords to filter and apply to relevant postings. Add features for skill-based ranking of jobs and integration with GitHub/LinkedIn profiles.
10	Dice–Focused Automated Job Application and Resume Keyword Scheduler	Build a system that reads resume skills and academic details to auto-apply for internships on Internshala. Include filters for domain, duration, and stipend, plus tracking of application status and interview invitations.
11	Monster–Focused Automated Job Application and Resume Keyword Scheduler	Develop a bot that matches resume keywords with Monster India job listings and automates profile submission and applications. Include scheduling, duplicate detection, and analytics on which keywords yield the most responses.
12	shine.com–Focused Automated Job Application and Resume Keyword Scheduler	Create an automation pipeline that extracts skills from resumes, searches shine.com for matching jobs, and auto-applies with cover letters. Provide a dashboard for students to view applied jobs, interview calls, and rejection reasons.
13	apna.com–Focused Automated Job Application and Resume Keyword Scheduler	Design a system that uses resume data to find and apply to hyperlocal jobs on apna.com, focusing on fresher-friendly roles. Include automated profile updates, job alerts, and application history tracking.

PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

14	Company Career Portal–Based Automated Job Application Framework	Build a generic framework that scrapes or integrates with company career pages to find openings matching a resume, then auto-fills and submits applications. Support multiple portals, handle varied form structures, and log submission confirmations.
15	AI based e-salesman, e-marketing executive system	Develop an AI agent that acts as a virtual sales and marketing executive, engaging customers via chat or voice, answering product queries, and guiding them through purchase decisions. Include lead scoring, personalized recommendations, and CRM integration.
16	LinkedIn and Social Media–Driven Automated Job Application System	Create a multi-platform tool that leverages LinkedIn, Twitter, and other social networks to identify job leads, network with recruiters, and auto-apply to postings. Focus on profile optimization, connection strategies, and application analytics.
17	End-to-End Faculty Recruitment, Interview, and Appointment Management System	Build a portal to manage the entire faculty hiring lifecycle: job posting, application collection, shortlisting, interview scheduling, feedback capture, and offer generation. Include role-based access for HR, committees, and candidates.
18	Student's interview experience, Interview Question Bank and Practice Platform	Develop a crowdsourced platform where students share interview experiences and questions, organized by company and role. Add mock interview features, AI-based question suggestions, and performance tracking for practice sessions.
19	Comprehensive Internship Management and Tracking System for Students	Create a system to manage internship applications, approvals, progress reports, and evaluations. Include features for mentor assignment, task tracking, feedback collection, and certificate generation upon completion.
20	face recognition based Smart Attendance Management System with Analytics and Reporting	Implement a face recognition system to mark student attendance in real time, with dashboards showing attendance trends, absentee alerts, and exportable reports. Ensure liveness detection to prevent spoofing and integrate with existing student databases.

PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

21	Integrated Capstone idea bank, capstone Project Management, Tracking, and Reporting Platform	Build a portal where students can browse capstone ideas, form teams, track milestones, submit reports, and receive faculty feedback. Include idea voting, plagiarism checks, and automated progress reminders.
22	AI-Powered Upskilling Recommendation and Automation System for Students	Develop an AI system that analyzes a student's profile (courses, projects, skills) and recommends personalized upskilling paths with curated resources, projects, and certifications. Track progress and adjust recommendations based on performance.
23	Automated Professional Network Connection and Engagement System	Create a tool that automatically identifies and connects with relevant professionals on LinkedIn and other networks, sends personalized messages, and tracks engagement. Include filters for industry, role, and mutual connections to avoid spam.
24	Internship Feedback Collection, Analysis, and Follow-Up System	Build a system to collect structured feedback from interns and mentors, analyze sentiment and themes, and generate actionable insights. Automate follow-up emails for missing feedback and produce summary reports for coordinators.
25	Pre-Admission corporate pitching Automation and Lead Management Platform	Develop a platform to automate corporate outreach for pre-admission internships or projects, including email campaigns, lead tracking, and meeting scheduling. Include templates for pitches and analytics on conversion rates.
26	AI-Driven Automated Networking and Connection System on Social Media	Implement an AI agent that identifies relevant professionals, engages with their content, and builds connections on social platforms. Focus on meaningful interactions, not mass messaging, with analytics on network growth.
27	Campus Lost-and-Found Portal with Reporting and	Create a web/mobile app where students can report lost items or found belongings, with image uploads, category tagging, and matching algorithms. Include admin moderation, notification alerts, and recovery confirmation workflows.

PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

	Recovery Workflow	
28	AI-Based Legal Assistant, e-lawyer and Case Guidance System	Build an AI chatbot that provides legal information, explains procedures, and guides users through common legal issues in India. Include document templates, case law references, and disclaimers that it is not a substitute for a lawyer.
29	AI-Mediated Phone Call System with Dynamic Translation and Transliteration	Develop a real-time phone call system where two parties speak in different languages, and AI translates and transliterates speech on the fly. Focus on low-latency audio processing, accurate translation, and natural voice output.
30	Voice-Command E-Doctor App for Symptom Triage and Care Coordination	Create a voice-enabled app where users describe symptoms, and the system provides triage advice, suggests specialists, and books appointments. Integrate with local clinics, maintain privacy, and include emergency escalation protocols.
31	Virtual Interview Audio Streaming and Hands-Free Communication App	Build an app that allows users to conduct virtual interviews or meetings using only audio, with noise cancellation and echo suppression. Include features for recording, transcription, and post-interview feedback.
32	AI-Enabled Smart Calculator with Voice, Camera, and Math OCR Capabilities	Develop a calculator app that accepts voice commands, captures math problems via camera, and solves them using OCR and symbolic computation. Support step-by-step solutions, graphing, and export of results.
33	Aerial Fencing and Corridor Management System for UAVs and Drones	Create a simulation or prototype system that defines virtual no-fly zones and safe corridors for drones using geofencing and real-time tracking. Include alerts for violations and integration with UAV navigation systems.
34	Food Safety App: Toxin Tracker, Health Impact, and Safer Alternatives	Build an app where users scan food labels or menus to get insights on additives, toxins, and health risks, with suggestions for safer alternatives. Include a database of ingredients, user reviews, and restaurant ratings.
35	Financial Scam Detection, Investigation, and	Develop a tool that analyzes transactions, messages, and calls to detect potential scams, alerts users, and guides them through reporting and recovery steps. Include educational content on common scam types and prevention tips.



PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

	Recovery Assistance System	
36	AI-Driven Global Surveillance and Intelligence Gathering Platform (e-RAW)	Create a conceptual or simulated platform that aggregates open-source intelligence (news, social media, satellite data) and uses AI to identify threats or trends. Focus on ethical use, data privacy, and compliance with laws.
37	EV Charging Station Finder with Slot Dashboard and Auto-Booking System	Build a mobile app that shows nearby EV charging stations, real-time slot availability, and allows users to book slots based on their vehicle's battery level. Include navigation, payment integration, and usage analytics.
38	Bulk Message Forwarding and Synchronization Tool: Telegram to WhatsApp	Develop a utility that forwards messages, media, and files from Telegram channels/groups to WhatsApp contacts or groups, with filters and scheduling. Ensure compliance with both platforms' terms of service.
39	Dehallucination Add-On Module for Enhancing Reliability of AI Applications	Create a module that detects and reduces hallucinations in AI-generated text by cross-referencing facts with trusted sources or knowledge graphs. Integrate with chatbots or content generators to improve accuracy.
40	Shopkeeper Announcement Mobile App Speaker替代 to Payment Machine Speakers	Build a mobile app that lets shopkeepers broadcast announcements, offers, or payment confirmations through their phone speaker, replacing dedicated payment machine speakers. Include customizable voice messages and volume control.
41	Doctor Prescription to Multilingual Video Reminder and Instruction Generator	Develop a system that converts doctor prescriptions into short, multilingual video reminders with dosage instructions and timing alerts. Support multiple Indian languages and integrate with pharmacy delivery apps.
42	Role-Specific AI Voice Assistant Framework for Organizations and Entities	Create a customizable AI voice assistant framework that can be tailored for specific roles (e.g., HR, IT support, admissions) within organizations. Include intent recognition, workflow automation, and integration with internal tools.



PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

43	Customer Propensity Modeling and Targeted Marketing Prediction System	Build a machine learning system that analyzes customer data to predict purchase propensity and recommend targeted marketing strategies. Include dashboards for campaign performance and ROI tracking.
44	Auto-Answer Phone System for Senior Citizens with Information Capture	Develop a phone system that automatically answers calls for senior citizens, captures key information (caller name, purpose), and notifies family members. Include voice prompts, spam filtering, and emergency contact escalation.
45	Face and Fingerprint–Based Customer Segmentation and Personalized Offer System	Create a retail system that uses biometric recognition to identify returning customers, segment them, and display personalized offers on digital screens. Ensure privacy compliance and opt-in consent mechanisms.
46	Sales Discussion Monitoring and Standardized Discount Recommendation System for POS	Build a tool that monitors sales conversations (via audio or text) at point-of-sale systems and suggests standardized discounts based on customer behavior and inventory levels. Include manager override and audit logs.
47	Priority-Based Medicine Booking and Allocation Module for Pharma Apps	Develop a module for pharmacy apps that prioritizes medicine bookings based on urgency, availability, and patient history. Include waitlist management, notifications, and integration with e-prescriptions.
48	AI-Optimized Railway Booking with Intermediate Berth and Route Combination Search	Create a system that searches all possible train routes, intermediate berths, and combinations to guarantee seat availability for long-distance travel. Include price comparison, booking automation, and cancellation handling.
49	Unified Online Admission Inquiry and Application Portal (Admission.com)	Build a centralized portal where students can explore colleges, compare courses, and submit admission applications to multiple institutions. Include chat support, document upload, and application status tracking.

PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

50	Anti–Performance Throttling Booster for Applications and Gaming Systems	Develop a software tool that detects and mitigates performance throttling in apps or games by optimizing system resources, background processes, and network usage. Include real-time monitoring and user-controlled profiles.
51	Streamlined AI-Assisted Research Paper Submission and Instant Publication Workflow	Create a platform that guides researchers through paper formatting, plagiarism checks, journal selection, and submission, with AI suggestions for improvements. Include templates for common journals and tracking of review status.
52	AI Traffic Police System for Violation Detection and Signal Optimization	Build a system using cameras and AI to detect traffic violations (speeding, red-light jumping) and dynamically optimize signal timings based on traffic flow. Include automated challan generation and integration with traffic databases.
53	Lost Mobile Tracking and Recovery System with Device Fingerprinting	Develop a tool that uses device fingerprints (IMEI, MAC address, installed apps) to track lost mobiles and assist in recovery. Include remote lock/wipe features and integration with law enforcement databases.
54	Monetary Fraud Investigation, Visualization, and Evidence Management Platform	Create a platform for investigators to visualize transaction networks, identify suspicious patterns, and manage evidence for fraud cases. Include timeline views, entity linking, and report generation.
55	Automated AI-Powered Student Mentoring and Progress Tracking System	Build an AI mentor that tracks student progress, suggests resources, and schedules check-ins based on performance and goals. Include sentiment analysis on feedback and early warning for at-risk students.
56	AI Traffic Assistant: Real-Time Help, Navigation, and Violation Alerts	Develop a mobile app that provides real-time traffic updates, route suggestions, and alerts for potential violations (speed limits, no-entry zones). Include voice guidance and integration with map services.

PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

57	Soldier Proximity Drone Detection and Situational Awareness System	Create a portable system for soldiers to detect nearby drones using RF or acoustic sensors, with alerts and countermeasure suggestions. Include GPS tagging and integration with command centers.
58	Counter–Anti-National Propaganda Narrative Detection and Mitigation Platform	Build an AI system that monitors social media and news for anti-national narratives, flags them for review, and suggests counter-messaging strategies. Include sentiment analysis, source credibility scoring, and reporting dashboards.
59	Face Recognition–Based Classroom Attendance System with Liveness Detection	Implement a face recognition attendance system that includes liveness detection to prevent photo/video spoofing. Integrate with student databases and generate attendance reports for faculty.
60	Student Attendance Management System	Develop a system to track student attendance manually or via biometrics, with automated reports, absentee alerts, and integration with academic calendars. Support role-based access for faculty and admins.
61	Online Bookstore	Build an e-commerce platform for buying and selling books, with user accounts, search filters, shopping cart, and payment gateway integration. Include admin panels for inventory and order management.
62	Hostel Management System	Create a system to manage hostel room allocations, student details, fee payments, and maintenance requests. Include waitlist management, room transfer workflows, and reporting for wardens.
63	Task Manager Application	Develop a task management tool with features like task creation, priority tagging, deadlines, reminders, and progress tracking. Support team collaboration with shared boards and comments.
64	Chat Application (Socket Programming)	Build a real-time chat application using WebSockets for instant messaging, with user authentication, private/group chats, and message history. Ensure scalability and low-latency communication.
65	Online Exam Portal	Create an online examination system with MCQ support, timer, auto-evaluation, and result generation. Include question banks, randomization, and proctoring features to prevent cheating.

PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

66	Event Management System	Develop a platform to manage events with registration, ticketing, scheduling, and participant communication. Include QR-based check-in, feedback collection, and analytics on attendance.
67	Smart Complaint Management System	Build a system for lodging, tracking, and resolving complaints with status updates, escalation workflows, and admin dashboards. Include category tagging, priority setting, and SLA monitoring.
68	Blood Bank Management System	Create a system to manage blood donation records, inventory levels, and request fulfillment. Include donor registration, expiry alerts, and integration with hospitals for emergency requests.
69	Vehicle Parking System	Develop a system to manage parking slots, vehicle entry/exit, and fee collection. Include RFID or license plate recognition, real-time slot availability, and monthly pass management.
70	Bank Management System	Build a system to handle core banking operations like deposits, withdrawals, balance inquiries, and transaction history. Include user authentication, role-based access, and audit logs.
71	Student Record System	Create a record-keeping system for student details, grades, attendance, and disciplinary actions. Support report generation, data export, and integration with other academic systems.
72	Hospital Management System	Develop a system for managing patient records, appointments, billing, and pharmacy inventory. Include doctor scheduling, lab test tracking, and discharge summaries.
73	Online Quiz System	Build a console or web-based quiz application with MCQs, scoring, and leaderboards. Support multiple categories, timed quizzes, and instant feedback on answers.
74	Railway Reservation System	Create a railway ticket reservation system with booking, cancellation, seat selection, and PNR status tracking. Include fare calculation, concession options, and integration with railway APIs.
75	Traffic Management System (Simulation)	Develop a simulation of traffic flow using data structures like queues and priority scheduling to model intersections and signal timings. Visualize congestion patterns and test optimization strategies.
76	File Compression Tool (Basic ZIP Implementation)	Build a basic ZIP compression tool using Huffman coding or similar algorithms to reduce file sizes. Include compression/decompression features and support for multiple file types.
77	Simple Shell (Command Line Interpreter)	Create a simple Unix-like shell that can execute basic commands (ls, cd, mkdir, etc.), handle pipes, and manage background processes. Focus on understanding OS internals and process management.

PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

78	Snake Game	Develop a classic Snake game using graphics libraries or console-based interfaces, with score tracking, increasing difficulty, and collision detection. Include options for custom grid sizes and speed settings.

Synopsis/Project Proposal

Each proposal should contain the following elements:

- Project title, Group number
- Team profile: Individual team members Name, SRN

Note: It is expected that every team member shall be involved in all project activities;

Proposed project description

Your project description must include a thorough explanation of your planned project, whether you are proposing a brand-new project or expanding on one of the sample software projects. You can also say whether you already have a customer or someone who is interested in your suggested project, as well as characterise the usual users of your planned system. Describe in a list of bullet points what the user will be able to perform with your system ("functional features") at the conclusion of this section.

Plan of work and product ownership:

- Your plan of work should list and describe the items that you are planning to accomplish in the short term (next few weeks).
- Split your team into "**who should do what** "
- Functionality: which functional features (from the itemized list at the end of the previous section) each person will contribute. *Example functional features are customer registration, data capture and storage, data processing to extract statistical parameters, etc.*



PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

- User interface, graphic design, database interaction, unit testing, etc. are not functional features.
- Qualitative property, if any, that you will contribute, such as fine tune the system performance to achieve response time under x seconds, or develop and evaluate an easy-to-use user interface for this-and-this specific functional feature, or ensure confidentiality of a specific set of data, etc.

Product ownership is critical to demonstrate that each team member will play a clearly defined role in the proposed project.

Evaluation

- Proposals that are missing any of the above sections will be returned without review.
- Each team should submit by given due date a single document for their team project.
- Only PDF document format will be accepted.

Jackfruit Phase-1

Software Requirements Specification

For <Project>

Version 1.0 approved



PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Team details

Team Name	**Project ID selected**	**Team Member 1 name and SRN per PES records**	**Team Member 2 name and SRN per PES records**	**Team Member 3 name and SRN per PES records**	**Team Member 4 name and SRN per PES records**

Prepared by <author>

<organization>

<date created>



PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Table of Contents

Table of Contents ii

Revision History ii

1. Introduction 1

1.1 Purpose 1

1.2 Intended Audience and Reading Suggestions 1

1.3 Product Scope 1

1.4 References 1

2. Overall Description 2

2.1 Product Perspective 2

2.2 Product Functions 2

2.3 User Classes and Characteristics 2

2.4 Operating Environment 2

2.5 Design and Implementation Constraints 2

2.6 Assumptions and Dependencies 3

3. External Interface Requirements 3

3.1 User Interfaces 3

3.2 Software Interfaces 3

3.3 Communications Interfaces 3

4. Analysis Models

5. System Features 4

5.1 System Feature 1 4

5.2 System Feature 2 (and so on) 4

6. Other Nonfunctional Requirements 4

6.1 Performance Requirements 4

6.2 Safety Requirements 5

6.3 Security Requirements 5

6.4 Software Quality Attributes 5

6.5 Business Rules 5

7. Other Requirements 5

Appendix A: Glossary 5

Appendix B: Field Layouts 5

Appendix C: Requirement Traceability matrix 6



PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Revision History

Name	Date	Reason For Changes	Version

Introduction

Purpose

<Identify the product whose software requirements are specified in this document, including the revision or release number. Describe the scope of the product that is covered by this SRS, particularly if this SRS describes only part of the system or a single subsystem.>

Intended Audience

<Describe the different types of reader that the document is intended for, such as developers, project managers, marketing staff, users, testers, and documentation writers. Describe what the rest of this SRS contains and how it is organized.

Product Scope

<Provide a short description of the software being specified and its purpose, including relevant benefits, objectives, and goals. Relate the software to corporate goals or business strategies. If a separate vision and scope document is available, refer to it rather than duplicating its contents here.>

References

<List any other documents or Web addresses to which this SRS refers. These may include user interface style guides, contracts, standards, system requirements specifications, use case documents, or a vision and scope document. Provide enough information so that the reader could access a copy of each reference, including title, author, version number, date, and source or location.>

Overall Description

Product Perspective

<Describe the context and origin of the product being specified in this SRS. For example, state whether this product is a follow-on member of a product family, a replacement for certain existing systems, or a new, self-contained product. If the SRS defines a component of a larger system, relate the requirements of the larger system to the functionality of this software and identify interfaces between the two. A simple diagram that shows the major components of the overall system, subsystem interconnections, and external interfaces can be helpful.>

Product Functions

<Summarize the major functions the product must perform or must let the user perform. Details will be provided in Section 3, so only a high level summary (such as a bullet list) is needed here. Organize the functions to make them understandable to any reader of the SRS. A picture of the major groups of related requirements and how they relate, such as a top level data flow diagram or object class diagram, is often effective.>

User Classes and Characteristics

<Identify the various user classes that you anticipate will use this product. User classes may be differentiated based on frequency of use, subset of product functions used, technical expertise, security or privilege levels, educational level, or experience. Describe the pertinent characteristics of each user class. Certain requirements may pertain only to certain user classes. Distinguish the most important user classes for this product from those who are less important to satisfy.>

Operating Environment

<Describe the environment in which the software will operate, including the hardware platform, operating system and versions, and any other software components or applications with which it must peacefully coexist.>

Design and Implementation Constraints

<Describe any items or issues that will limit the options available to the developers. These might include: corporate or regulatory policies; hardware limitations (timing requirements, memory requirements); interfaces to other applications; specific technologies, tools, and databases to be used; parallel operations; language requirements; communications protocols; security considerations; design conventions or programming standards (for example, if the customer's organization will be responsible for maintaining the delivered software).>

2.6 Assumptions and Dependencies

<List any assumed factors (as opposed to known facts) that could affect the requirements stated in the SRS. These could include third-party or commercial components that you plan to use, issues around the development or operating environment, or constraints. The project could be affected if these assumptions are incorrect, are not shared, or change. Also identify any dependencies the project has on external factors, such as software components that you intend to reuse from another project, unless they are already documented elsewhere (for example, in the vision and scope document or the project plan).>

External Interface Requirements

User Interfaces

<Describe the logical characteristics of each interface between the software product and the users. This may include sample screen images, any GUI standards or product family style guides that are to be followed, screen layout constraints, standard buttons and functions (e.g., help) that will appear on every screen, keyboard shortcuts, error message display standards, and so on. Define the software components for which a user interface is needed. Details of the user interface design should be documented in a separate user interface specification.>

Software Interfaces

<Describe the connections between this product and other specific software components (name and version), including databases, operating systems, tools, libraries, and integrated commercial components. Identify the data items or messages coming into the system and going out and describe the purpose of each. Describe the services needed and the nature of communications. Refer to documents that describe detailed application programming interface protocols. Identify data that will be shared across software components. If the data sharing mechanism must be implemented in a specific way (for example, use of a global data area in a multitasking operating system), specify this as an implementation constraint.>

Communications Interfaces

<Describe the requirements associated with any communications functions required by this product, including e-mail, web browser, network server communications protocols, electronic forms, and so on. Define any pertinent message formatting. Identify any communication standards that will be used, such as FTP or HTTP. Specify any communication security or encryption issues, data transfer rates, and synchronization mechanisms.>

<if any Hardware Interfaces include as 3.4 Hardware Interfaces>

Analysis Models

<Include pertinent analysis models, such as use case diagrams and if applicable entity-relationship diagrams.>

System Features

<This template illustrates organizing the functional requirements for the product by system features, the major services provided by the product. You may prefer to organize this section by use case, mode of operation, user class, object class, functional hierarchy, or combinations of these, whatever makes the most logical sense for your product.>

System Feature 1

<Don't really say "System Feature 1." State the feature name in just a few words.>

5.1.1 Description and Priority

<Provide a short description of the feature and indicate whether it is of High, Medium, or Low priority. You could also include specific priority component ratings, such as benefit, penalty, cost, and risk (each rated on a relative scale from a low of 1 to a high of 9).>

5.1.2 Stimulus/Response Sequences

<List the sequences of user actions and system responses that stimulate the behavior defined for this feature. These will correspond to the dialog elements associated with use cases.>

5.1.3 Functional Requirements

<Itemize the detailed functional requirements associated with this feature. These are the software capabilities that must be present in order for the user to carry out the services provided by the feature, or to execute the use case. Include how the product should respond to anticipated error conditions or invalid inputs. Requirements should be concise, complete, unambiguous, verifiable, and necessary. Use "TBD" as a placeholder to indicate when necessary information is not yet available.>

<Each requirement should be uniquely identified with a sequence number or a meaningful tag of some kind.>

REQ-1:
REQ-2:

System Feature 2 (and so on)

Other Nonfunctional Requirements

Performance Requirements

<If there are performance requirements for the product under various circumstances, state them here and explain their rationale, to help the developers understand the intent and make suitable design choices. Specify the timing relationships for real time systems. Make such requirements as specific as possible. You may need to state performance requirements for individual functional requirements or features.>

Safety Requirements

<Specify those requirements that are concerned with possible loss, damage, or harm that could result from the use of the product. Define any safeguards or actions that must be taken, as well as actions that must be prevented. Refer to any external policies or regulations that state safety issues that affect the product's design or use. Define any safety certifications that must be satisfied.>

Security Requirements

<Specify any requirements regarding security or privacy issues surrounding use of the product or protection of the data used or created by the product. Define any user identity authentication requirements. Refer to any external policies or regulations containing security issues that affect the product. Define any security or privacy certifications that must be satisfied.>

Software Quality Attributes

<Specify any additional quality characteristics for the product that will be important to either the customers or the developers. Some to consider are: adaptability, availability, correctness, flexibility, interoperability, maintainability, portability, reliability, reusability, robustness, testability, and usability. Write these to be specific, quantitative, and verifiable when possible. At the least, clarify the relative preferences for various attributes, such as ease of use over ease of learning.>

Business Rules

<List any operating principles about the product, such as which individuals or roles can perform which functions under specific circumstances. These are not functional requirements in themselves, but they may imply certain functional requirements to enforce the rules.>

Also include Domain requirements here.

Other Requirements

<Define any other requirements not covered elsewhere in the SRS. This might include database requirements, internationalization requirements, legal requirements, reuse objectives for the project, and so on. Add any new sections that are pertinent to the project.>

Appendix A: Glossary

<Define all the terms necessary to properly interpret the SRS, including acronyms and abbreviations. You may wish to build a separate glossary that spans multiple projects or the entire organization, and just include terms specific to a single project in each SRS.>

Appendix B: Field Layouts

An Excel sheet containing field layouts and properties/attributes and report requirements.

Sample sheet with information required to register the customer

Field	Length	Data Type	Description	Is Mandatory
Account Number	16	Numeric		Y
ISFC code	11	Alphanumeric		Y
Card Amount	20	Numeric		Y
Mandate Start Date	8	Date	Date of Mandate Registration	N
Mandate End Date	8	Date	Date of Mandate Expiry	N
Status	25	Alphanumeric	Status of Registration	Y
Customer Name	60	String		Y

Reject Reason Code	4	String	Reject Reason code in case mandate is rejected	N

Sample Report Requirements: Include the fields to be included in the report

Registration Report	**Transaction Report**
Bank Account Number	Transaction Reference Number
ISFC Code	Bank Account Number
Bank Name	IFSC Code
Account Status	Bank Name
Account Type	Customer Name
Customer Name	Card Number
Card Number	Debit Transaction Amount
SI Start Date	Transaction Date
Status	Status
Remarks	Debit Attempt Number
	Remarks

Appendix C: Requirement Traceability Matrix

Sl. No	Requirement ID	Brief Description of Requirement	Architecture Reference	Design Reference	Code File Reference	Test Case ID	System Test Case ID

Jackfruit Phase-2

Project Plan Document

Instructions:

1: Prepare a detailed plan for your project which comprises of the below mentioned details.

2: Upload pdf document through the given link.

3: The name of the document should be your Project ID.

Things to be included as part of the project plan.

1: Identify the lifecycle to be followed for the execution of your project and justify why you have chosen the model.

2: Identify the tools which u want to use it throughout the lifecycle like planning tool, design tool, version control, development tool, bug tracking, testing tool.

3: Determine all the deliverables and categorise them as reuse/build components and justify the same.

4: Create a WBS for the entire functionalities in detail.

5: Do a rough estimate of effort required to accomplish each task in terms of person months.

6: Create the Gantt Chart for scheduling using any tool.

Jackfruit Phase-3

Test Plan Document

Instructions:

1: Prepare at least 8-10 test cases for each **implemented use case** (functional requirements) as per the below given template which should include Unit, Integration and System Test cases.

2: Carry out manual testing for all the test cases and populate the columns Actual Result and Test Result.

Template of a Test case:

Test Case ID	Name of Module	Test case description	Pre-conditions	Test Steps	Test data	Expected Results	Actual Result	Test Result

- **Test Case ID :** Each test case should be represented by a unique ID. To indicate test types, follow some convention like "UT_01" indicating "Unit Testing - Test Case#1."
- **Name of the module :** Specify the name of the **main module or sub module** being tested
- **Test Case Description :** Specify the summary or test purpose in brief
- **Pre- Conditions :** Any requirement that needs to be done before execution of this test case.
- **Test Steps** : Mention all the steps in detail and specify the order in which it is to be executed.
- **Test Data :** Input for the test case to be executed. Specify different data sets with precise values to be used as input. (create test case for both valid and invalid inputs)
- **Expected Results :** Mention the expected results including error or precise messages that should be displayed on screen
- **Actual Results :** After execution of test case fill this column with the result obtained
- **Test Result (Pass/Fail) :** Mark this field as “fail” if the actual result is not same as expected result else mark as “pass”.

Sample Test Case:

Test Case ID	Name of Module	Test case description	Pre-conditions	Test Steps	Test data	Expected Results	Actual Result	Test Result
UT-01	User registration module	To test the login functionality	Access to Chrome Browser	1: Navigate to http://www.demo.com 2: Enter Username and Password 3: Click Submit	User name: PESU Student. Password : pes123	Login should be successful with “welcome message”	Login successful with “welcome message” displayed	Pass

Jackfruit Submission

Final Report & Demonstration



B.TECH. (CSE)

V SEMESTER

UE24CS341A –SOFTWARE ENGINEERING

PROJECT REPORT

ON

PROJECT TITLE HERE

SUBMITTED BY

NAME	SRN
1) Student Name	
2) Student Name	
3) Student Name	

August – Nov 2022

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
BENGALURU – 560100, KARNATAKA, INDIA

TABLE OF CONTENTS

Sl.No	TOPIC	PAGE No
1.	PROPOSAL OF THE PROJECT	
2.	SOFTWARE REQUIREMENTS SPECIFICATION	
3.	PROJECT PLAN	
4.	DESIGN DIAGRAMS	
5.	TEST CASES	
6.	SCREEN SHOTS OF OUTPUT	